

DA6400: Reinforcement Learning

Programming Assignment 1

Team Members: EE23B085, EE23B067, CH23B122

Drive Link: https://drive.google.com/file/d/1g2EIa0fIQ5YBhkHV_QhmkLYjmzyzsdV/view?usp=sharing

Work Contributions:

EE23B085 (Eswar) — Question 1: Grid World

EE23B067, CH23B122 — Question 2: Acrobot

March 5, 2026

1. 🤖 MDP Design for Windy Forest

3M

Question

Design the MDP tuple (S, A, P, R, γ) . Show transition and reward matrices for cell $(3, 3)$ and its eight neighbors. Take $\gamma = 0.95$.

Solution

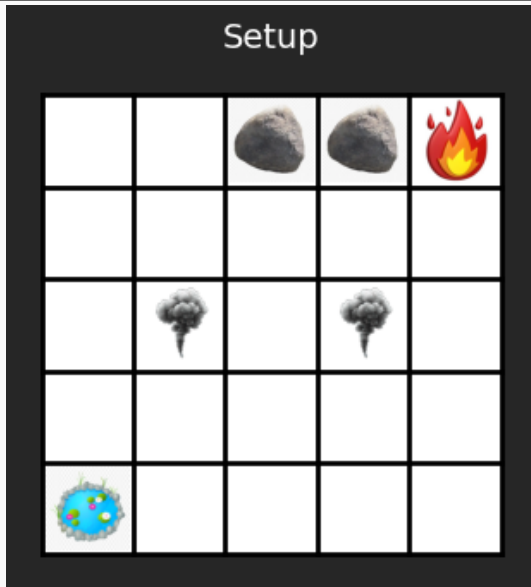


Figure 1: Grid-world setup

Expected Reward: $R(s, a) = \sum_{s'} P(s, a, s') \cdot r(s, s')$ stored in MDP model.

Transition Probabilities: Normal cells: $P_{\text{intended}} = 0.7$, $P_{\text{stay}} = 0.1$, $P_{\text{perp}} = 0.1$ each. Smoke cells: $P_{\text{intended}} = 0.4$, $P_{\text{stay}} = 0.4$, $P_{\text{perp}} = 0.1$ each. Hover $\Rightarrow P_{\text{stay}} = 1$. Off-grid $\Rightarrow$ stay. Boulders & fire+water are absorbing.

State Space: $S = (x, y, w)$, $x, y \in \{0..4\}$, $w \in \{0, 1\}$

$\Rightarrow |S| = 50$

Action Space: $A = \{N, S, E, W, \text{Hover}\}$

Discount: $\gamma = 0.95$

Rewards $r(s, a, s') = R(s') \forall a$, so $r(s, s') = R(s')$:

Landing cell	Reward
Fire ($w=1$)	+100
Boulder	-100
Smoke	-11
Other	-1

Around (3,3) - Transition Probabilities

	(2,2)	(3,2)	(4,2)	(2,3)	(3,3)	(4,3)	(2,4)	(3,4)	(4,4)
North	0	0	0	0.1	0.1	0.1	0	0.7	0
South	0	0.7	0	0.1	0.1	0.1	0	0	0
East	0	0.1	0	0	0.1	0.7	0	0.1	0
West	0	0.1	0	0.7	0.1	0	0	0.1	0
Hover	0	0	0	0	1	0	0	0	0

Around (3,3, w = 1) - Rewards from (3,3)

	(3,4) Smoke_Fire	
(2,3) Free_Water	(3,3) Free_Water	(4,3) Free_Water
	(3,2) Smoke_Water	

Probabilities are identical for $w = 0$ and $w = 1$ since neither lake nor fire are in reach.

2. 🗺️ Optimal Value Function & Corresponding Policy

5M

Question

Value iteration code and Visualize the optimal value function and the corresponding policy [5M]

Solution

Code: [core/Agent/](#). States split into two phases by water level ($w = 0, 1$). Policy is greedy w.r.t. V^* (converged in ~ 176 iterations).

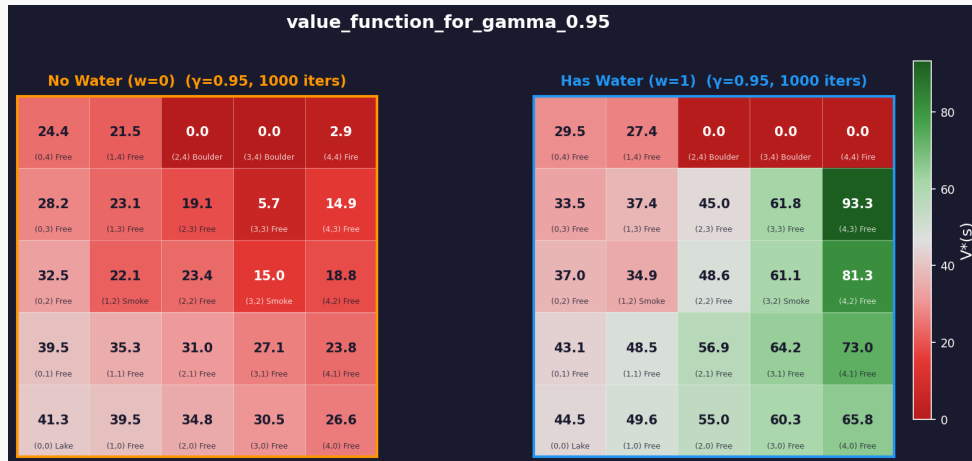


Figure 2: Optimal value function $V^*(s)$ for both phases: **No Water** ($w = 0$, navigate to lake) and **Has Water** ($w = 1$, navigate to fire). Color gradient shows expected cumulative reward.

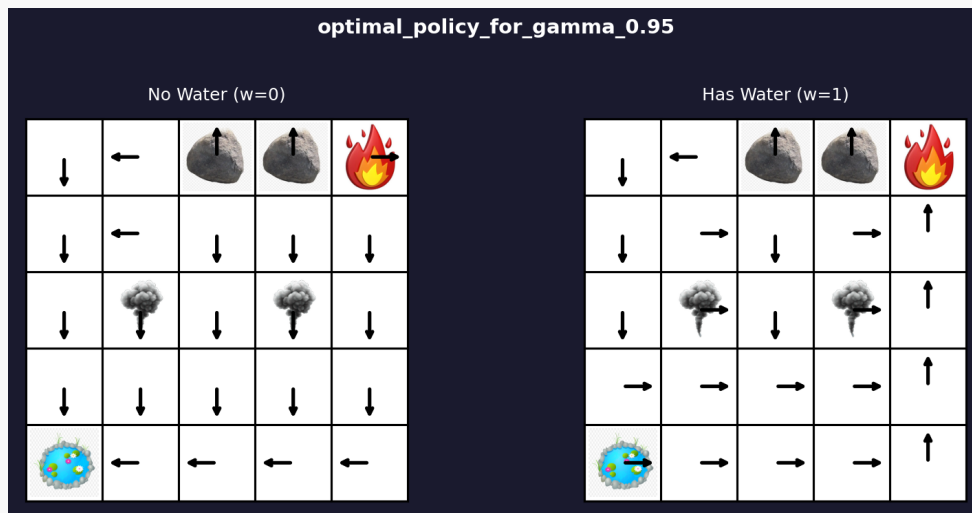


Figure 3: Optimal policy $\pi^*(s)$ showing greedy actions for all cells in both phases. Arrows indicate the best direction (N/S/E/W/Hover) to maximize expected return from each state.

3. ⚙️ MDP with $\gamma = 0.3$

9M

Question

Suppose we change the above MDP by setting $\gamma = 0.3$. [9M]

(a) Optimal Value Function & Policy

With the original MDP ($R(s_{\text{boulder}}, a) = 0$, absorbing), the greedy policy at $\gamma = 0.3$ drives the drone **into** boulders:

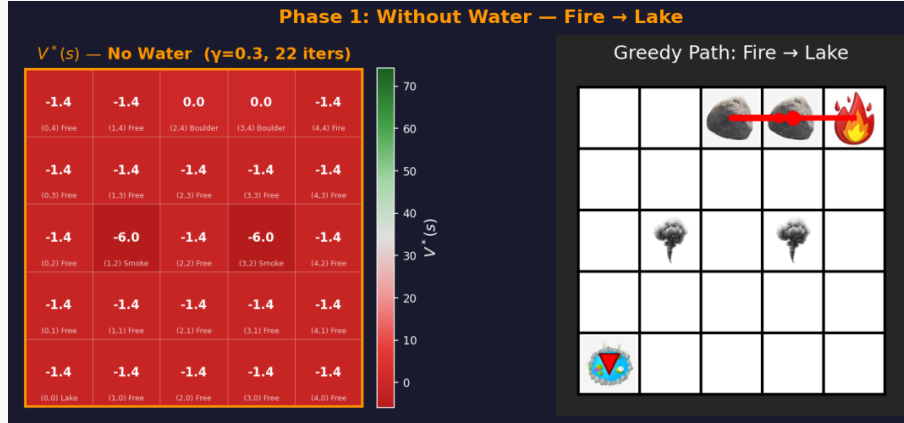


Figure 4: $\gamma = 0.3$ with original model — drone crashes into boulders.

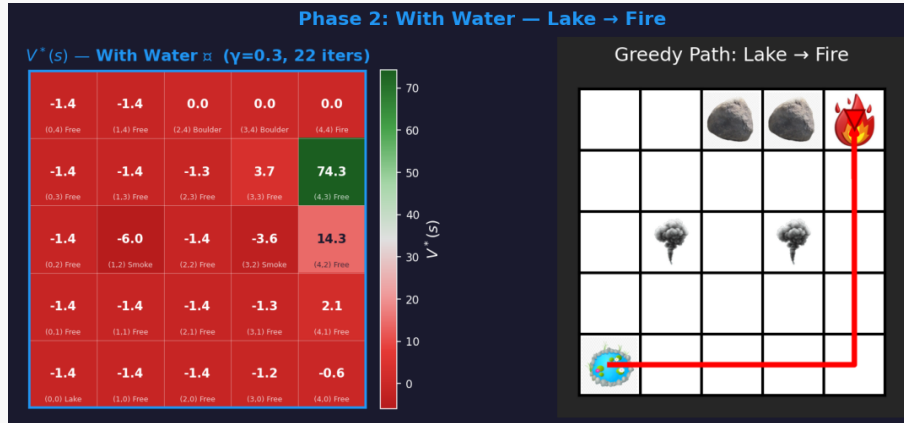


Figure 5: $\gamma = 0.3$ with original model moving towards fire with $w = 1$

Why? With low γ the agent is *myopic*: the future reward of reaching the lake (~ 8 steps away) is discounted by $0.3^8 \approx 0.00066$, making it negligible. Meanwhile every step costs -1 , so the boulder ($V = 0$) looks *better* than continuing ($V < 0$). The agent “prefers suicide over living.”

Fix — two model changes:

1. Set $R(s_{\text{boulder}}, a) = -100$ (non-zero absorbing reward) $\Rightarrow V(s_{\text{boulder}}) = \frac{-100}{1-\gamma}$, making boulders strongly negative.
 2. Set $r(s, s_{\text{lake}}) = +100$ when $w = 0$ — gives the drone a *closer* positive signal to pursue.
- After these changes the drone avoids boulders and learns a viable two-phase policy:

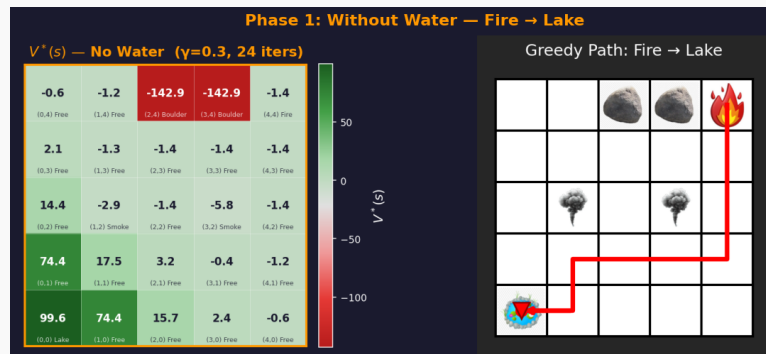


Figure 6: Modified model, $\gamma = 0.3$ — Phase 1: Fire → Lake.

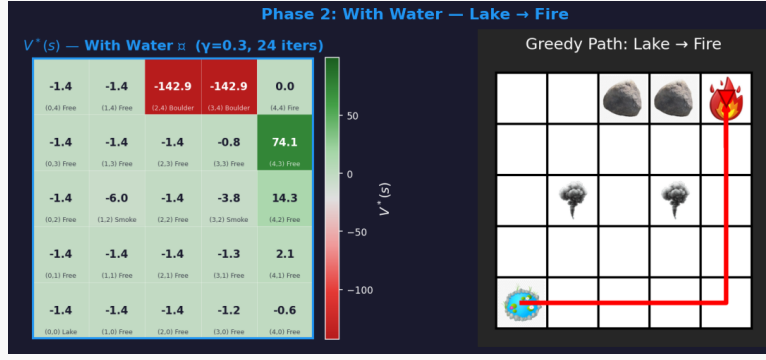


Figure 7: Modified model, $\gamma = 0.3$ — Phase 2: Lake → Fire.

Compared to $\gamma = 0.95$: values are much smaller in magnitude (heavy discounting), and the policy is less “globally aware” — it reacts mostly to nearby rewards.

(b) Near Hazardous States

In **both** $\gamma = 0.95$ and $\gamma = 0.3$, the value function near smoke cells $\{(1,2), (3,2)\}$ is *lower* than surrounding cells — the -11 penalty makes them locally unattractive regardless of γ .

The key difference: at $\gamma = 0.95$, the values near hazards are still **positive** (e.g. $V \approx 20$ – 40) because the discounted future reward from reaching fire/lake propagates strongly. At $\gamma = 0.3$, the values near hazards are **negative** — the agent is myopic, so the distant $+100$ reward is discounted by 0.3^k (negligible for $k > 3$), and all it “sees” is the accumulated -1 step penalties and -11 smoke penalties, yielding $V < 0$ everywhere.

(c) Hovering Preference

Yes, at $\gamma = 0.3$, the drone *does* hover at certain locations surrounded by hazards:

- $(2,2)$ with $w \in \{0,1\}$ — between the two smoke cells $\{(1,2), (3,2)\}$
- $(3,3)$ with $w = 0$ — between the boulder cells $\{(2,4), (3,4)\}$

Why hover? With $\gamma = 0.3$, the agent is extremely *myopic* — it only looks ~ 3 – 4 steps ahead. When a cell is surrounded by hazards (smoke or boulders), all neighboring moves incur immediate penalties (-11 or risk of -100), while the distant reward is heavily discounted: $0.3^k \times 100 \approx 0$ for $k > 5$. In this regime, hovering ($V_{\text{hover}} = \frac{-1}{1-0.3} \approx -1.43$) becomes the *least bad* option — the drone prefers to “do nothing” rather than risk a hazardous move for negligible future gain.

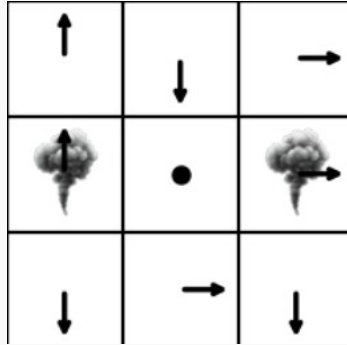


Figure 8: $\gamma = 0.3$: The optimal policy shows hovering (no arrow) at $(2,2)$ and $(3,3, w=0)$ — cells surrounded by hazards where short-term risk outweighs distant rewards.

At $\gamma = 0.95$, hovering is *never* optimal: $V_{\text{hover}} = -20$, while moving toward the goal yields $V \gg -20$ (e.g. $V \approx 30$ – 60), so the drone always prefers action over inaction.

(d) Asymmetric Smoke-Cell Values

An interesting asymmetry emerges at smoke cell $(3,2)$ depending on water level:

-1.4 (1,3) Free	-1.4 (2,3) Free	-1.4 (3,3) Free	-1.4 (4,3) Free
-6.0 (1,2) Smoke	-1.4 (2,2) Free	-6.0 (3,2) Smoke	-1.4 (4,2) Free
-1.4 (1,1) Free	-1.4 (2,1) Free	-1.4 (3,1) Free	-1.4 (4,1) Free

Figure 9: Smoke (3,2) with $w = 0$: both smoke cells have *equal* values — distance to lake is similar.

-1.4 (1,3) Free	-1.3 (2,3) Free	3.7 (3,3) Free	74.3 (4,3) Free
-6.0 (1,2) Smoke	-1.4 (2,2) Free	-3.6 (3,2) Smoke	14.3 (4,2) Free
-1.4 (1,1) Free	-1.4 (2,1) Free	-1.3 (3,1) Free	2.1 (4,1) Free

Figure 10: Smoke (3,2) with $w = 1$: (3,2) has *higher* value than (1,2) — closer to fire.

Why this happens:

- **With water ($w = 1$):** Cell (3,2) is ~ 3 steps from fire (4,4), yielding discounted reward $0.3^3 \times 100 \approx 2.7$ — still *significant*. In contrast, (1,2) is farther (~ 5 steps), so $0.3^5 \times 100 \approx 0.24$ — much weaker signal. The agent clearly “sees” that (3,2) is closer to the goal.
- **Without water ($w = 0$):** Both smoke cells are ~ 12 – 14 steps from lake (0,0). The discount factor obliterates the difference: $0.3^{12} \approx 5 \times 10^{-7}$ and $0.3^{14} \approx 5 \times 10^{-8}$ — both effectively zero. The agent is “blind” to which cell is closer; it only perceives the immediate -11 penalty, making their values identical. This demonstrates how *myopia* at low γ collapses distant distinctions: when goals are more than ~ 5 steps away, the agent cannot differentiate between paths — all futures look equally bleak.

4. Ξ MDP with Increased Hazard Penalty

10M

Question

Suppose we change the MDP by penalizing the drone more for entering hazardous states (penalty = 90). Keep $\gamma = 0.95$. [10M]

Solution

(a) Optimal Value Function & Policy with -90 Hazard Penalty

With -90 smoke penalty and $\gamma = 0.9$, smoke regions become *negative convergence zones*.

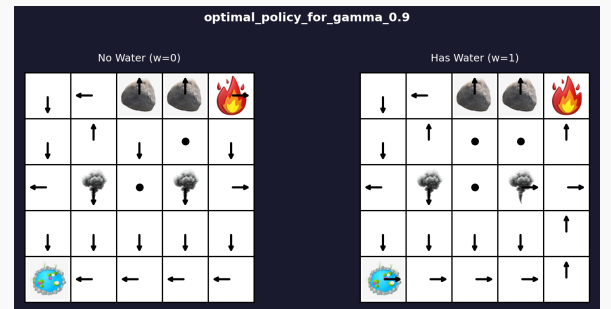
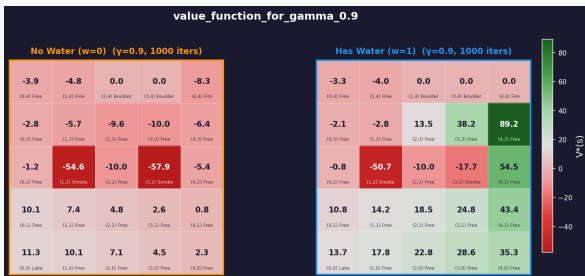


Figure 11: Value function and policy ($\gamma = 0.9$, hazard = -90).

Key observations: (i) Smoke nearly as bad as boulders; (ii) (4,2) avoids despite 2-step proximity due to drift; (iii) Slower convergence—balanced immediate/future penalties; (iv) Planning: $\gamma = 0.3 \rightarrow 1.4$ steps, $0.9 \rightarrow 10$, $0.95 \rightarrow 20$.

(b) Hovering Scenarios

Yes, surprisingly $\gamma = 0.9$ has **5/50 hovering states** (most of all): (3,3) and (4,3) with $w \in \{0,1\}$; (1,3) with $w = 0$.

“Indecision Threshold”: At $\gamma = 0.3$, shortsighted—best action obvious (minimal hovering). At $\gamma = 0.95$, farsighted—clear long-term strategy (minimal hovering). At $\gamma = 0.9$, immediate/future rewards have *equal weight*, creating ambiguity. Q-values for competing actions nearly identical; Bellman updates “waver.” Result: maximum hovering. Paradox: more hovering than myopic $\gamma = 0.3$ because caught between timescales, not from shortsightedness.

(c) Longer Path Preference

Yes. Example: from smoke (1, 2), the optimal policy takes a long detour (see Fig.) rather than the direct 3-step path to fire (4, 4).

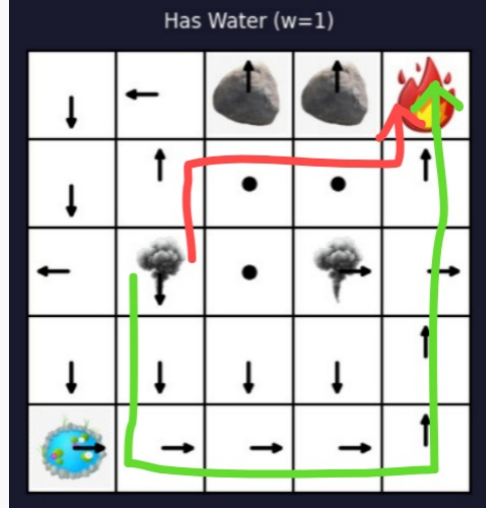


Figure 12: Long detour from (1, 2) to avoid hazards.

Reason: The short path passes through 2 boulders $\{(2, 4), (3, 4)\}$ and smoke (3, 2). With stochastic wind, cumulative hazard-hit probability is too high. The long path—even using off-grid bouncebacks—minimizes -90 smoke and -100 boulder risks. When hazard penalties are severe, safety beats efficiency.

(d) Stronger Wind Analysis

Stronger wind ($P_{\text{fwd}} = 0.4$, $P_{\text{perp}} = 0.25$ each, $P_{\text{stay}} = 0.1$) degrades control significantly:



Figure 13: Value function and policy with stronger wind ($\gamma = 0.95$, hazard = -90).

Key observations:

- Increased stochasticity:** Perpendicular drift rises from 10% to 25% ($2.5\times$ higher)—movement becomes much less predictable.
- Control degradation:** Forward probability drops from 70% to 40% ($\downarrow 43\%$); total perpendicular risk rises from 20% to 50% ($\uparrow 150\%$). Result: *half* of all movements now go sideways.
- Lower value functions:** Expected cumulative rewards decrease across all states due to higher chance of unintended hazard encounters.
- More conservative policy:** Agent prefers wider safety margins around hazards, more hovering in ambiguous locations, longer detour paths to avoid risk corridors.
- Convergence behavior:** Took more iterations due to increased uncertainty in Bellman updates. Combined with -90 smoke penalties, this creates “no-go zones” where smoke regions become near-impassable. Models storm degradation scenarios where control authority severely compromised.

TD-based Control in Acrobot

The remainder of this report moves from the Grid-world MDP to the classic *Acrobot* control task; the question numbering below is restarted for this new environment.

1. Python code for SARSA and Q-learning

Question

Question 1: Write Python code for SARSA and Q-learning using ϵ -greedy exploration in the above environment. What is the number of states?

Solution

The state consists of: $\sin(\theta_1), \cos(\theta_1), \sin(\theta_2), \cos(\theta_2), \dot{\theta}_1, \dot{\theta}_2$. The goal is to swing the tip above a specific height.

Number of States: Each value in the 6-dimensional observation vector is binned with 10 bins, so there are a total of 10^6 states. Since there are 3 possible actions for each state, the size of the Q-Table is 3×10^6 .

Summary of Experiments: Q-learning demonstrated better performance over SARSA when tested for 5 episodes for each algorithm.

Code is present in the repo

2. Reward Scheme Comparison

Question

Question 2: Plot return vs. timesteps/episodes curves comparing SARSA and Q-learning. [10M]

Solution

(a) Plots for Top 3 Hyperparameters for SARSA and Q-Learning

We trained on 45 identical sets of hyperparameters (step size α , exploration rate ϵ) for both algorithms.

Is a constant ϵ sufficient for exploration?

- **Constant ϵ :** Results in conservative value estimates due to persistent exploration, limiting the final policy performance.
- **ϵ -decay:** Allows sufficient early exploration while gradually reducing randomness, leading to improved convergence and higher final returns.

From the graphs, even for the top 3 parameters across Q-learning and SARSA, an average reward of around -300 to -200 (over 10,000 steps) was obtained, which is characteristic of an average learning agent. Thus, we applied an ϵ -decay to improve the overall reward, using the best set of hyperparameters obtained previously. We observed only a slight improvement in the final reward since we started with an already low ϵ value and the experiment only ran for 10,000 steps.

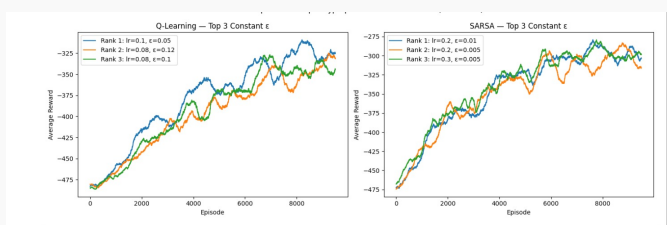


Figure 14: Top 3 hyperparameters (SARSA/Q-Learning)

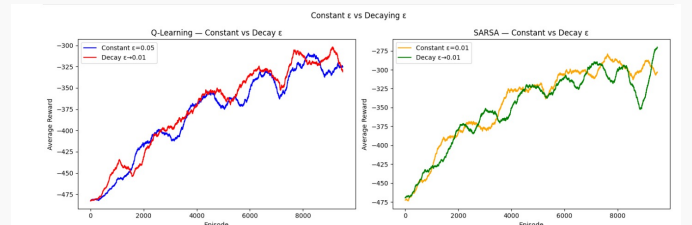


Figure 15: Constant ϵ vs ϵ -decay

(b) Performance Benchmarking across Multiple Random Seeds

From the graph, we can see that Q-learning performs slightly better than SARSA and the confidence bands for Q-learning are smaller compared to SARSA, indicating that the former is more stable compared to the latter.

One possible reason could be the nature of the two algorithms themselves. Q-learning is an off-policy algorithm, learning the optimal policy independently of the agent's actions, while SARSA is an on-policy algorithm. This makes SARSA more sensitive to the ongoing exploration behavior, leading to higher variance and performance differences caused by the distinct random seeds.

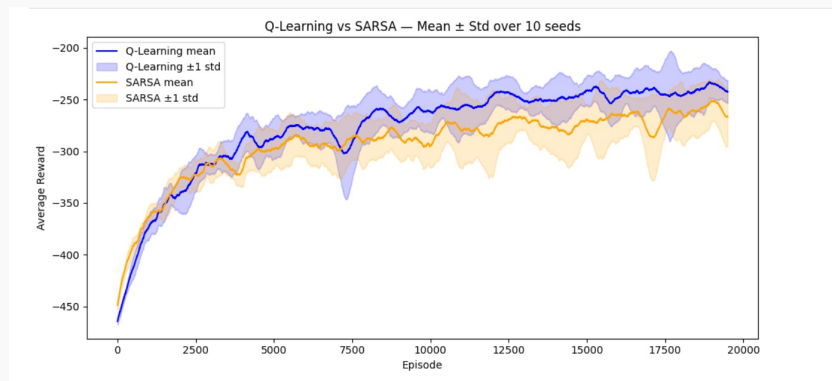


Figure 16: SARSA vs Q-Learning across 10 random seeds

3. Decaying Exploration and Performance Comparison

Question

Question 3: Run the two algorithms starting with $\epsilon = 1$, decaying ϵ till 0.1, and fixed thereafter. Compare (i) the online performance (performance while learning) and (ii) the performance of the policies after finishing learning (i.e., without any exploration). Explain the differences, if there exist any. [5M]

Solution

(a) Online Performance (During Learning)

During training, online performance is naturally lower and noisier due to the ϵ -greedy exploration strategy introducing suboptimal exploratory actions. Initially, SARSA tends to handle the high exploration rate slightly better since it is an on-policy algorithm and accounts for the ongoing exploration in its updates. In contrast, Q-learning, being off-policy, learns greedily and initially behaves more randomly. However, as the number of steps progresses and ϵ decays, Q-learning improves rapidly and exhibits lower variance. Ultimately, both algorithms converge to similar performance levels, with variations largely resulting from the applied state discretization and the environment's random initial starting conditions.

(b) Final Performance ($\epsilon = 0$)

After training, disabling exploration ($\epsilon = 0$) leads to significantly improved and highly stable performance, as the agents no longer execute random suboptimal actions. In this purely exploitative phase, Q-learning typically achieves higher final returns down the line due to its off-policy nature, which has been independently learning the optimal greedy path all along. On the other hand, SARSA—having learned a policy that inherently incorporates the risk of the ϵ -greedy exploration—naturally converges to a safer, more conservative policy.

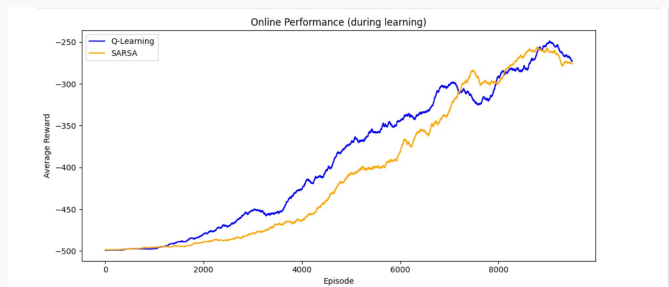


Figure 17: Online performance (ϵ -decay)



Figure 18: Final performance ($\epsilon = 0$)

4. Design Insights and Bonus

Question

Question 4: Intuitively, increasing the number of bins results in better state representation, enhancing granularity. This can result in learning a better policy. Is this always the case? Are there any downsides? Explain (try 5, 15, and 20 bins). [3M]

Solution

Is increasing the number of bins always beneficial?

No. Increasing the number of bins increases the cardinality of the discretized state space exponentially, leading to the **curse of dimensionality**. While finer discretization reduces approximation bias (allowing for more accurate policy mapping), it dramatically increases sample complexity. Insufficient visitation of state-action pairs for a large number of bins leads to slower convergence, poorer empirical performance within a limited number of steps, and higher policy variance. More bins do not inherently guarantee better rewards; the overall performance remains heavily dependent on tuned hyperparameters and the total number of training episodes.

Effect of different bin sizes:

- **5 bins:** Coarse discretization with high bias. Learning is fast, but the model groups distinctly different environmental states together. This hinders actual learning by confusing the nature of optimal actions, leading to a highly suboptimal, unsteady, and oscillatory policy (especially visible in SARSA error propagation).
- **15 bins & 20 bins:** Improved granularity but significantly larger state space. While acting as a better representation, learning becomes slower and more unstable. For a constrained number of steps (e.g., 10,000 steps), the algorithm struggles because many granular state-action pairs are rarely visited. Learning becomes practically inefficient with standard tabular methods due to severely sparse state-action coverage.

Downsides summary: Beyond a moderate optimal number of bins (such as 10 bins, which often outperforms higher bin setups), the theoretical benefit of finer representation is outweighed by slower convergence, increased variance, and error propagation. There exists an optimal intermediate bin size that perfectly balances the bias-variance tradeoff.

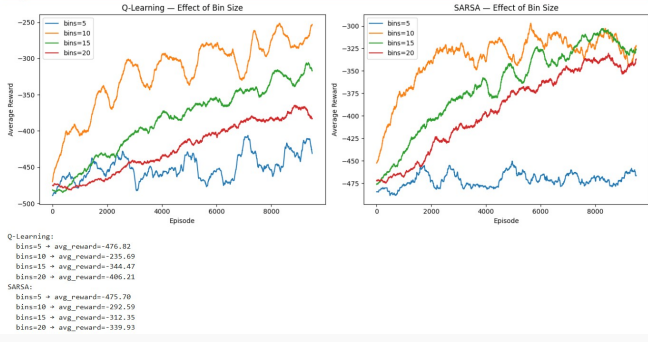


Figure 19: Effect of bins on Q-learning

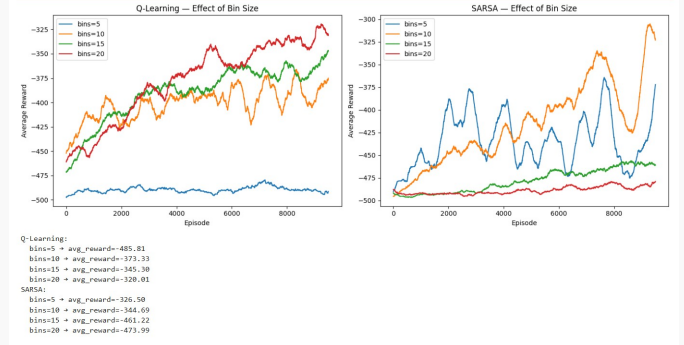


Figure 20: Effect of bins on SARSA

5. Modified Reward Setting

Question

Question 5: Suppose we modify the reward setting to a per-step reward as given by Eq. 1:

$$r = \frac{\eta h}{2} + \text{sign}(-1 + \eta h) \left(\frac{2 - \eta h}{2} \right), \quad \eta > 0$$

where h is the height of the tip. Answer the following questions theoretically. [12M]

Solution

Mathematical Setup of the Reward: Simplifying the reward piecewise around the threshold $h^* = \frac{1}{\eta}$:

- If $\eta h < 1$ (i.e., $h < 1/\eta$): $r(h) = \eta h - 1$.
- If $\eta h \geq 1$ (i.e., $h \geq 1/\eta$): $r(h) = 1$.

(a) Learning Speed and Unexpected Behavior (4M)

The modified reward provides a **dense and monotonic shaping signal**. For $h < h^*$, the gradient $dr/dh = \eta > 0$ is constant. This dense shaping improves temporal credit assignment, which typically leads to **faster learning** compared to the classic sparse (-1 until success) reward.

However, it can induce **unexpected behaviors** (reward hacking). The sharp saturation at h^* introduces a discontinuity capable of creating spurious local optima. The agent may learn a policy to rapidly swing up and constantly hover just above the threshold h^* to repeatedly exploit the $+1$ reward, rather than completing the full continuous task. This results in heavy oscillations or stalling near locally rewarding heights.

(b) Behavior when $\eta = 0.5$ (2M)

For $\eta = 0.5$, the saturation threshold becomes $h^* = 1/0.5 = 2$. The reward effectively reduces to $r(h) = 0.5h - 1$ for the entire operational range (since the maximum height is 2). This remains completely negative for all strictly valid states prior to the goal. Because the agent receives larger relative penalties initially, early learning might be slightly **slower**; however, the positive slope still provides excellent, steady guidance directly encouraging the agent to increase height all the way to $h = 2$.

(c) Behavior for $\eta \in \{1, 2, 5\}$ and recommended range (3M)

Increasing η directly heightens the reward sensitivity to height by steepening the gradient ($dr/dh = \eta$), accelerating early learning but rapidly decreasing the saturation threshold $h^* = 1/\eta$. This drastically increases the risk of exploitative hovering and oscillations.

- $\eta = 1$ ($h^* = 1$): Moderate shaping providing balanced mid-way guidance.
- $\eta = 2$ ($h^* = 0.5$): Low saturation threshold. Strongly encourages partial lifts but heavily risks converging to incomplete solutions (hovering halfway).

- $\eta = 5$ ($h^* = 0.2$): Extreme saturation threshold. Promotes trivial oscillations and total exploitation of minor height gains.

Recommended Range: $\eta \in [0.5, 1]$. This range perfectly balances an informative, dense gradient for shaping without misaligning the reward thresholds, thereby preventing reward exploitation and preserving task fidelity while improving learning efficiency.

(d) Insights on Reward Design (3M)

- **Shaping vs. Policy Fidelity:** Dense rewards (shaping) improve convergence speed but easily distort the optimal policy if scaled poorly. The saturation threshold $h^* = 1/\eta$ acts as a tradeoff limit; shaping must be scaled so that h^* lies within a meaningful, task-relevant progress region ($\eta \approx 1/h^*$).
- **Smoothness & Invariance:** Effective reward design should preserve original task equivalence. Discontinuities and poorly placed saturation points act as misleading local optima. Well-designed rewards remain smooth, provide informative gradients, and strictly align with the ultimate task objective rather than intermediate trivialities.